Advanced Public Economics — Applied Tutorial

Session VI: Microsimulation Models II

Daniel Weishaar

Roadmap for Session VI

① Very brief introduction to essential Python commands.

② Use a Python-based microsimulation model for Germany to:

- understand the complexity of the German tax and transfer system,
- obtain a realistic picture of effective marginal tax rates (EMTRs),
- decompose EMTRs into their underlying policy components.

Python/GETTSIM — Installation Instructions (Windows / macOS)

① Install Anaconda (Windows/macOS).

② Open a command line:

- **Windows:** *Anaconda Prompt*
- **macOS:** *Terminal*

③ Create a clean environment (recommended to avoid conflicts):

3.1 `conda create -n microsimulation python=3.11 -y`

3.2 `conda activate microsimulation`

④ Install GETTSIM and Jupyter **inside this environment**:

4.1 `conda install -c conda-forge gettsim jupyter -y`

⑤ Quick test:

5.1 `python -c "import gettsim; print(gettsim.__version__)"`

⑥ Start Jupyter (after activating the environment):

6.1 `jupyter notebook`

Python — Getting Started

Running Python

GETTSIM is a Python package consisting of multiple modules. We run Python code using the Jupyter Notebook environment.

- Interactive execution: write code and immediately see results.
- Combines *code, output, and explanations* in one document.
- Widely used for teaching, data analysis, and research.

Python — Getting Started

Running Python

GETTSIM is a Python package consisting of multiple modules. We run Python code using the Jupyter Notebook environment.

- Interactive execution: write code and immediately see results.
- Combines *code, output, and explanations* in one document.
- Widely used for teaching, data analysis, and research.

Python packages

Similar to libraries in R, Python packages extend functionality. We mainly use:

- *pandas*: data handling and analysis,
- *numpy*: numerical computations,
- *gettsim*: tax-benefit microsimulation for Germany.

Python — Getting Started

Running Python

GETTSIM is a Python package consisting of multiple modules. We run Python code using the Jupyter Notebook environment.

- Interactive execution: write code and immediately see results.
- Combines *code, output, and explanations* in one document.
- Widely used for teaching, data analysis, and research.

Python packages

Similar to libraries in R, Python packages extend functionality. We mainly use:

- *pandas*: data handling and analysis,
- *numpy*: numerical computations,
- *gettsim*: tax-benefit microsimulation for Germany.

$\Rightarrow$ Some packages are pre-installed; others (like *gettsim*) must be installed manually.

Python — Basic Objects

Core data types

- **Numbers**: integers and floats
  - `x = 5,  y = 2.5`
- **Strings**: text objects
  - `name = "Daniel"`
- **Lists**: ordered collections
  - `ages = [25, 30, 45]`
- **Dictionaries**: key–value mappings
  - `person = {"age":  30, "income":  40000}`

Python — Basic Objects

Core data types

- **Numbers**: integers and floats
  - `x = 5,  y = 2.5`
- **Strings**: text objects
  - `name = "Daniel"`
- **Lists**: ordered collections
  - `ages = [25, 30, 45]`
- **Dictionaries**: key–value mappings
  - `person = {"age":  30, "income":  40000}`

Tabular data

- **pandas DataFrame**: main object for data analysis.
- Comparable to a data frame or tibble in R.

Python — Basic Commands

Assignment and inspection

- Assign objects using =
  - `income = 50000`
- Check object type:
  - `type(income)`
- Display object contents:
  - `print(income)`

Python — Basic Commands

Assignment and inspection

- Assign objects using =
  - `income = 50000`
- Check object type:
  - `type(income)`
- Display object contents:
  - `print(income)`

Working with packages

- Import a package:
  - `import pandas as pd`
- Access objects inside a package:
  - `pd.DataFrame(...)`
- Get help:
  - `help(pd.DataFrame)`

Python — Pandas Package — Data Frames

What is a DataFrame?

- A **DataFrame** is a table with rows and columns
- Implemented in Python via the `pandas` package
- Direct analogue to a data frame (or tibble) in R

Python — Pandas Package — Data Frames

What is a DataFrame?

- A **DataFrame** is a table with rows and columns
- Implemented in Python via the `pandas` package
- Direct analogue to a data frame (or tibble) in R

Creating a DataFrame

- Most common: from a dictionary (columns = keys)

```
df = pd.DataFrame({
 "id": [1, 2, 3],
 "income": [20000, 35000, 50000],
 "age": [25, 40, 55],
})
```

- Rows = individuals (or households), Columns = characteristics and incomes

Python — NumPy Package — Numerical Computations

- **Create numerical arrays**

```
import numpy as np
x = np.array([20000, 35000, 50000])
```

Python — NumPy Package — Numerical Computations

- **Create numerical arrays**

```
import numpy as np
x = np.array([20000, 35000, 50000])
```

- **Vectorized computations (no loops)**

```
tax_rate = 0.3
tax = tax_rate * x
```

Python — NumPy Package — Numerical Computations

- **Create numerical arrays**

```
import numpy as np
x = np.array([20000, 35000, 50000])
```

- **Vectorized computations (no loops)**

```
tax_rate = 0.3
tax = tax_rate * x
```

- **Basic statistics**

```
np.mean(x), np.median(x), np.percentile(x, 90)
```

Python — NumPy Package — Numerical Computations

- **Create numerical arrays**

```
import numpy as np
x = np.array([20000, 35000, 50000])
```

- **Vectorized computations (no loops)**

```
tax_rate = 0.3
tax = tax_rate * x
```

- **Basic statistics**

```
np.mean(x), np.median(x), np.percentile(x, 90)
```

- **Create grids (useful for EMTR profiles)**

```
income_grid = np.linspace(0, 100000, 100)
```

$\Rightarrow$ Hands-on exercise: import packages, generate objects.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

- In contrast to TAXSIM, computations and code are transparent and open source.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

- In contrast to TAXSIM, computations and code are transparent and open source.
- As in TAXSIM, status quo tax systems for different years are pre-programmed.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

- In contrast to TAXSIM, computations and code are transparent and open source.
- As in TAXSIM, status quo tax systems for different years are pre-programmed.
- But, in contrast to TAXSIM, we can manipulate parameters of tax systems to simulate counterfactuals, e.g., an increase in the unemployment insurance contribution rate.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

- In contrast to TAXSIM, computations and code are transparent and open source.
- As in TAXSIM, status quo tax systems for different years are pre-programmed.
- But, in contrast to TAXSIM, we can manipulate parameters of tax systems to simulate counterfactuals, e.g., an increase in the unemployment insurance contribution rate.

Comparison to TAXSIM

Last session, we used the NBER TAXSIM microsimulation model to estimate effective marginal tax rates for the US. ⇒ Today, we use GETTSIM for Germany.

- In contrast to TAXSIM, computations and code are transparent and open source.
- As in TAXSIM, status quo tax systems for different years are pre-programmed.
- But, in contrast to TAXSIM, we can manipulate parameters of tax systems to simulate counterfactuals, e.g., an increase in the unemployment insurance contribution rate.

⇒ Computations follow a specific **Directed Acyclic Graphs (DAG)** structure.

GETTSIM — Structure — Directed Acyclic Graph (DAG)

GETTSIM represents the tax–benefit system as a **directed acyclic graph (DAG)**.

Simple Example: Gross income $\rightarrow$ Income tax $\rightarrow$ Net income

What does this mean?

- **Nodes** : objects, e.g., economic variables (e.g. gross income, taxes, transfers).
- **Edges** ($\rightarrow$): logical dependencies between variables.
- **Directed**: variables are computed in a well-defined order, i.e., no $\leftrightarrow$ relationships.
- **Acyclic**: no circular dependencies.

GETTSIM — Structure — Directed Acyclic Graph (DAG)

GETTSIM represents the tax–benefit system as a **directed acyclic graph (DAG)**.

Simple Example: Gross income $\rightarrow$ Income tax $\rightarrow$ Net income

What does this mean?

- **Nodes** : objects, e.g., economic variables (e.g. gross income, taxes, transfers).
- **Edges** ($\rightarrow$): logical dependencies between variables.
- **Directed**: variables are computed in a well-defined order, i.e., no $\leftrightarrow$ relationships.
- **Acyclic**: no circular dependencies.

Why this matters

- Ensures internal consistency of the tax–transfer system.
- Allows automatic recomputation when inputs change.
- Makes policy reforms transparent and modular.

GETTSIM — Structure — Visualizing Tax and Transfer System (I)

DAG structure of tax-transfer system can be visualized in gettsim (Hands-on exercise).

Full System:

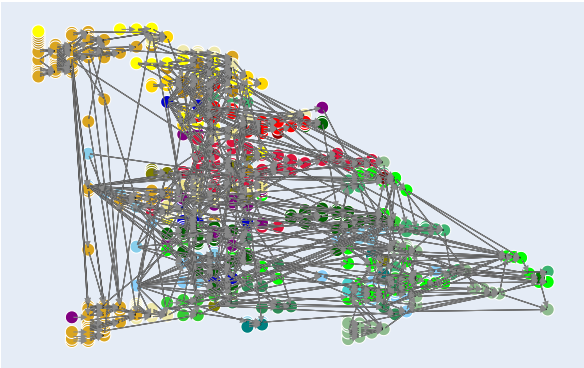
```
plot.dag.tt(
  policy_date_str="2025-01-01",
  width=1200,
  height=800,
  show_node_description = True
)
```

Parts of System:

```
plot.dag.tt(
  policy_date_str="2025-01-01", # policy date
  primary_nodes={"sozialversicherung__beiträge_versicherter_m"}, # node of interest
  selection_type="ancestors", # look only backwards from the node
  selection_depth=1, # look only one step backward
  show_node_description = True # show description of nodes
)
```

GETTSIM — Structure — Visualizing Tax and Transfer System (II)

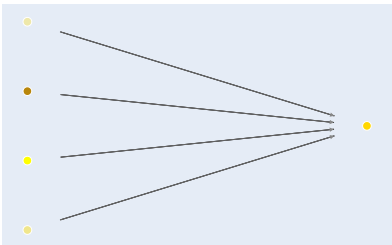
Full System:



- Colors indicate background/general input variables, intermediate earnings (*Einkünfte*), social insurance programs, transfer programs, and taxes.
- Detailed color descriptions via `plot.dag.tt?`.

GETTSIM — Structure — Visualizing Tax and Transfer System (III)

Partial System:



We can also exclusively look at ancestors, neighbors, or descendants. Here, we look at ...

- ... ancestors (`selection_type="ancestors"`),
- ... only direct ones (`selection_depth=1`),
- ... for particular node (`primary_nodes={"sozialversicherung__beiträge_versicherter_m"}`).

⇒ We see that the total social insurance contributions (right node) are direct ancestors of four nodes on the left, i.e., the components of social insurance contributions (unemployment, pension, long term care, health).

GETTSIM — Structure — Inspect Nodes

DAG gives high-level overview of relationships. But how are nodes related?

① Find object associated with a policy environment and a node name.
② Observe type of object (function, input, parameter).
③ Inspect characteristics of object, e.g., to see how a function works.

⇒ Hands-on exercise

GETTSIM — Structure — Inspect Nodes

DAG gives high-level overview of relationships. But how are nodes related?

① Find object associated with a policy environment and a node name.
② Observe type of object (function, input, parameter).
③ Inspect characteristics of object, e.g., to see how a function works.

⇒ Hands-on exercise

Example Inspection: total social insurance contributions

```
@policy_function()
 def beiträge_versicherter_m(
   pflege__beitrag__betrag_versicherter_m: float,
   kranken__beitrag__betrag_versicherter_m: float,
   rente__beitrag__betrag_versicherter_m: float,
   arbeitslosen__beitrag__betrag_versicherter_m: float,
 ) -> float:
 """Sum of social insurance contributions paid by the insured person."""
 return (
   pflege__beitrag__betrag_versicherter_m
   + kranken__beitrag__betrag_versicherter_m
   + rente__beitrag__betrag_versicherter_m
   + arbeitslosen__beitrag__betrag_versicherter_m
 )
```

GETTSIM — Structure — Inspect Nodes

DAG gives high-level overview of relationships. But how are nodes related?

① Find object associated with a policy environment and a node name.
② Observe type of object (function, input, parameter).
③ Inspect characteristics of object, e.g., to see how a function works.

⇒ Hands-on exercise

Example Inspection: total social insurance contributions

```
@policy_function()
 def beiträge_versicherter_m(
   pflege__beitrag__betrag_versicherter_m: float,
   kranken__beitrag__betrag_versicherter_m: float,
   rente__beitrag__betrag_versicherter_m: float,
   arbeitslosen__beitrag__betrag_versicherter_m: float,
 ) -> float:
 """Sum of social insurance contributions paid by the insured person."""
 return (
   pflege__beitrag__betrag_versicherter_m
   + kranken__beitrag__betrag_versicherter_m
   + rente__beitrag__betrag_versicherter_m
   + arbeitslosen__beitrag__betrag_versicherter_m
 )
```

GETTSIM — Simulate Policy

Having observed the structure of (some) policies, we can now simulate them.

Steps:

1. Define targets to simulate (e.g., tax, social insurance contribution, transfers, ...).
2. Check input variables necessary to simulate policy.
3. Generate data for hypothetical household with all necessary input variables.
4. Simulate targets.

GETTSIM — Simulate Policy

Having observed the structure of (some) policies, we can now simulate them.

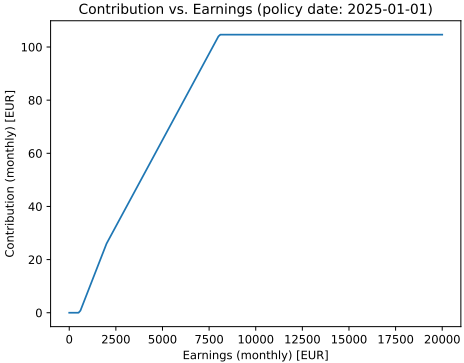
Steps:

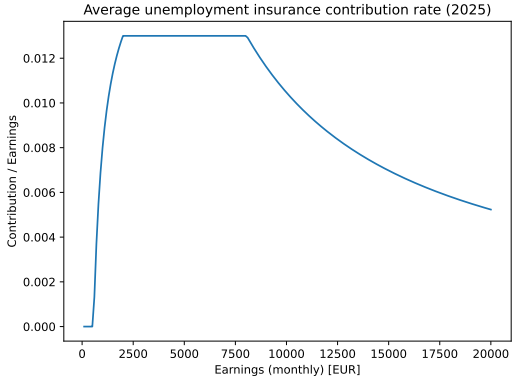
1. Define targets to simulate (e.g., tax, social insurance contribution, transfers, ...).
2. Check input variables necessary to simulate policy.
3. Generate data for hypothetical household with all necessary input variables.
4. Simulate targets.

⇒ Hands-on exercise:

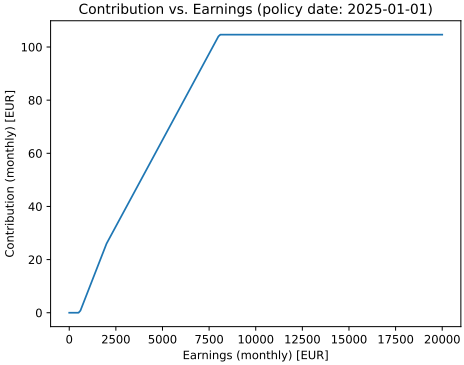
- Calculate for hypothetical household the contribution for unemployment insurance.
- Show how contribution for unemployment insurance varies across income.

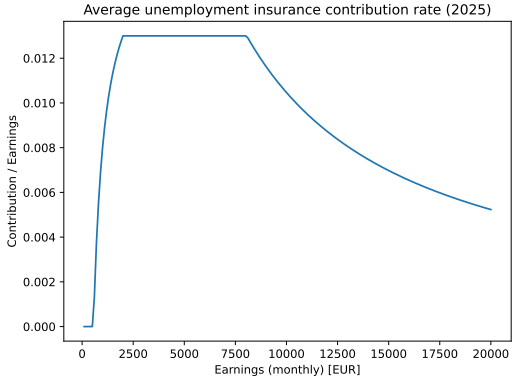
GETTSIM — Simulate Policy — Unemployment Insurance Contribution





GETTSIM — Simulate Policy — Unemployment Insurance Contribution



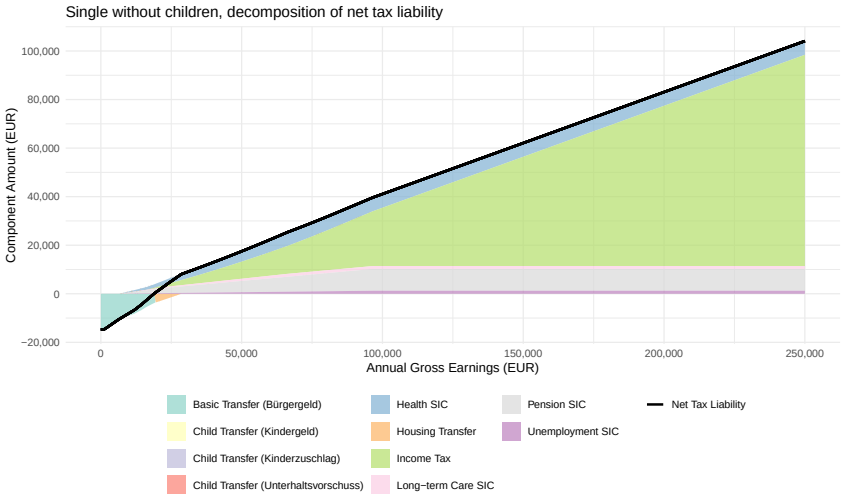


- Unemployment insurance contribution just one of many programs.
- Now simulate taxes and transfers (requires many more input variables).

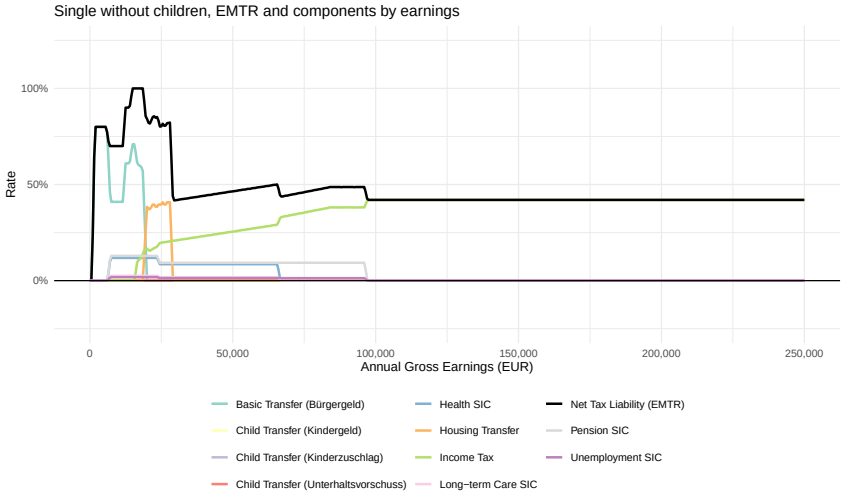
$\Rightarrow$ Hands-on exercise

GETTSIM — Simulate Status Quo Taxes and Transfers — Net Tax

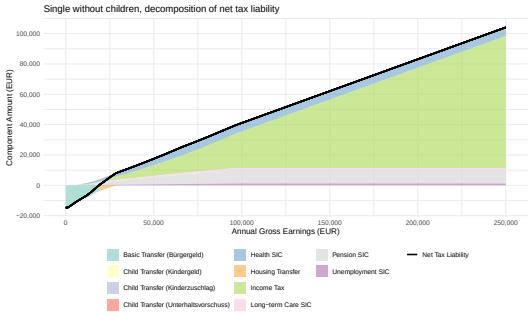
Here, look at single household without children (as for United States with TAXSIM):

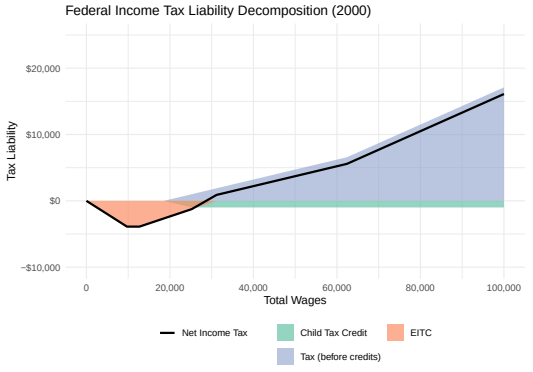


GETTSIM — Simulate Status Quo Taxes and Transfers — EMTR



Tax and Transfer System — Germany vs. United States





- In Germany, basic transfer starts directly with zero earnings, and is then phased out at very high rates.
- In the United States, no basic transfer. EITC is phased in gradually, and is then phased out at much lower rates.

Microsimulation and Effective Marginal Tax Rates — Bigger Picture

⇒ Why is all this now relevant for public finance analyses?

- When we estimated optimal tax rates in session 5 for Germany, we incorporated only a limited view on the status quo tax system (statutory marginal tax rates).
- Optimal rates characterized by u-shape, and thus high rates at the bottom, while actual (statutory) rates were low at the bottom and monotonically increasing.
- EMTR much closer to a u-shape than statutory MTR.

⇒ **Maybe the German system not so bad after all** (considering the full system)?

⇒ Next session, use EMTR schedule to study **tax reforms**!

That's it! See you next time!